

# Softmax exercise

Complete and hand in this completed worksheet (including its outputs and any supporting code outside of the worksheet) with your assignment submission. For more details see the [assignments page](#) on the course website.

This exercise is analogous to the SVM exercise. You will:

- implement a fully-vectorized **loss function** for the Softmax classifier
- implement the fully-vectorized expression for its **analytic gradient**
- **check your implementation** with numerical gradient
- use a validation set to **tune the learning rate and regularization** strength
- **optimize** the loss function with **SGD**
- **visualize** the final learned weights

```
In [11]: # Update ipython version (requires restart)
!pip install ipython==8.1.0
```

```
In [12]: ### IF YOU ARE USING COLAB, UNCOMMENT AND RUN THIS BLOCK FIRST ###

# # Mount google drive to allow access to your files
# from google.colab import drive
# drive.mount('/content/drive')
# drive_folder = '/content/drive/MyDrive'
# # Adjust this line to be the assignment1 folder in your google drive
# notebook_folder = drive_folder + '/cs682/assignment1'
# %cd {notebook_folder}
```

```
In [13]: from __future__ import print_function
import random
import numpy as np
from cs682.data_utils import load_CIFAR10
import matplotlib.pyplot as plt

%matplotlib inline
plt.rcParams['figure.figsize'] = (10.0, 8.0) # set default size of plots
plt.rcParams['image.interpolation'] = 'nearest'
plt.rcParams['image.cmap'] = 'gray'

# for auto-reloading external modules
# see http://stackoverflow.com/questions/1907993/autoreload-of-modules-in-ipy
%load_ext autoreload
%autoreload 2
```

The autoreload extension is already loaded. To reload it, use:  
%reload\_ext autoreload

```
In [14]: def get_CIFAR10_data(num_training=49000, num_validation=1000, num_test=1000,
    """
    Load the CIFAR-10 dataset from disk and perform preprocessing to prepare
    it for the linear classifier. These are the same steps as we used for the
    SVM, but condensed to a single function.
    """
    # Load the raw CIFAR-10 data
    cifar10_dir = 'cs682/datasets/cifar-10-batches-py'
    # Don't forget to run get_datasets.sh, or this will throw an error
    X_train, y_train, X_test, y_test = load_CIFAR10(cifar10_dir)

    # subsample the data
    mask = list(range(num_training, num_training + num_validation))
    X_val = X_train[mask]
    y_val = y_train[mask]
    mask = list(range(num_training))
    X_train = X_train[mask]
    y_train = y_train[mask]
    mask = list(range(num_test))
    X_test = X_test[mask]
    y_test = y_test[mask]
    mask = np.random.choice(num_training, num_dev, replace=False)
    X_dev = X_train[mask]
    y_dev = y_train[mask]

    # Preprocessing: reshape the image data into rows
    X_train = np.reshape(X_train, (X_train.shape[0], -1))
    X_val = np.reshape(X_val, (X_val.shape[0], -1))
    X_test = np.reshape(X_test, (X_test.shape[0], -1))
    X_dev = np.reshape(X_dev, (X_dev.shape[0], -1))

    # Normalize the data: subtract the mean image
    mean_image = np.mean(X_train, axis = 0)
    X_train -= mean_image
    X_val -= mean_image
    X_test -= mean_image
    X_dev -= mean_image

    # add bias dimension and transform into columns
    X_train = np.hstack([X_train, np.ones((X_train.shape[0], 1))])
    X_val = np.hstack([X_val, np.ones((X_val.shape[0], 1))])
    X_test = np.hstack([X_test, np.ones((X_test.shape[0], 1))])
    X_dev = np.hstack([X_dev, np.ones((X_dev.shape[0], 1))])

    return X_train, y_train, X_val, y_val, X_test, y_test, X_dev, y_dev

# Cleaning up variables to prevent loading data multiple times (which may cause
try:
    del X_train, y_train
    del X_test, y_test
    print('Clear previously loaded data.')
except:
    pass
```

```
# Invoke the above function to get our data.
X_train, y_train, X_val, y_val, X_test, y_test, X_dev, y_dev = get_CIFAR10_data()
print('Train data shape: ', X_train.shape)
print('Train labels shape: ', y_train.shape)
print('Validation data shape: ', X_val.shape)
print('Validation labels shape: ', y_val.shape)
print('Test data shape: ', X_test.shape)
print('Test labels shape: ', y_test.shape)
print('dev data shape: ', X_dev.shape)
print('dev labels shape: ', y_dev.shape)
```

```
Clear previously loaded data.
Train data shape: (49000, 3073)
Train labels shape: (49000,)
Validation data shape: (1000, 3073)
Validation labels shape: (1000,)
Test data shape: (1000, 3073)
Test labels shape: (1000,)
dev data shape: (500, 3073)
dev labels shape: (500,)
```

## Softmax Classifier

Your code for this section will all be written inside **cs682/classifiers/softmax.py**.

```
In [15]: # First implement the naive softmax loss function with nested loops.
# Open the file cs682/classifiers/softmax.py and implement the
# softmax_loss_naive function.

from cs682.classifiers.softmax import softmax_loss_naive
import time

# Generate a random softmax weight matrix and use it to compute the loss.
W = np.random.randn(3073, 10) * 0.0001
loss, grad = softmax_loss_naive(W, X_dev, y_dev, 0.0)

# As a rough sanity check, our loss should be something close to -log(0.1).
print('loss: %f' % loss)
print('sanity check: %f' % (-np.log(0.1)))
```

```
loss: 2.421284
sanity check: 2.302585
```

## Inline Question 1:

Why do we expect our loss to be close to  $-\log(0.1)$ ? Explain briefly.\*\*

**Your answer:** In the lecture notation the scores are  $z_j = W_j x_i$ , the softmax gives

$$p(y = i | x) = \frac{e^{z_i}}{\sum_{j=1}^K e^{z_j}}, \text{ and the loss is } L_i = -\log\left(\frac{e^{z_i}}{\sum_{j=1}^K e^{z_j}}\right).$$

$W$  is initialised with very small random values ( $\times 0.0001$ ), so every  $z_j \approx 0$ . Then each  $e^{z_j} \approx 1$  and, with  $K = 10$  classes,  $p(y = y_i | x) \approx \frac{1}{10} = 0.1$ . Therefore  $L_i \approx -\log(0.1) = \log 10 \approx 2.303$ , and the average  $L = \frac{1}{N} \sum_i L_i$  (no regularization here) is the same. This is a sanity check that the loss is implemented correctly.

```
In [16]: # Complete the implementation of softmax_loss_naive and implement a (naive)
# version of the gradient that uses nested loops.
loss, grad = softmax_loss_naive(W, X_dev, y_dev, 0.0)

# As we did for the SVM, use numeric gradient checking as a debugging tool.
# The numeric gradient should be close to the analytic gradient.
from cs682.gradient_check import grad_check_sparse
f = lambda w: softmax_loss_naive(w, X_dev, y_dev, 0.0)[0]
grad_numerical = grad_check_sparse(f, W, grad, 10)

# similar to SVM case, do another gradient check with regularization
loss, grad = softmax_loss_naive(W, X_dev, y_dev, 5e1)
f = lambda w: softmax_loss_naive(w, X_dev, y_dev, 5e1)[0]
grad_numerical = grad_check_sparse(f, W, grad, 10)
```

```
numerical: 0.266113 analytic: 0.266113, relative error: 4.079215e-09
numerical: 0.681800 analytic: 0.681800, relative error: 7.974048e-08
numerical: -1.648941 analytic: -1.648942, relative error: 1.269473e-08
numerical: -2.280319 analytic: -2.280319, relative error: 4.440655e-09
numerical: -1.839154 analytic: -1.839154, relative error: 2.194938e-08
numerical: -0.242307 analytic: -0.242307, relative error: 2.612713e-07
numerical: -2.911632 analytic: -2.911632, relative error: 3.675620e-09
numerical: -0.635279 analytic: -0.635279, relative error: 1.256604e-07
numerical: 1.231575 analytic: 1.231575, relative error: 1.280162e-08
numerical: -4.010223 analytic: -4.010223, relative error: 4.999984e-09
numerical: 3.438571 analytic: 3.438571, relative error: 1.813460e-08
numerical: -0.165316 analytic: -0.165316, relative error: 6.082173e-08
numerical: 0.685930 analytic: 0.685930, relative error: 3.243538e-08
numerical: -0.408935 analytic: -0.408935, relative error: 1.069412e-08
numerical: -2.012154 analytic: -2.012154, relative error: 1.479997e-08
numerical: 0.942949 analytic: 0.942949, relative error: 2.542433e-08
numerical: 2.716527 analytic: 2.716527, relative error: 7.432499e-09
numerical: 5.152193 analytic: 5.152193, relative error: 1.083130e-08
numerical: -1.137575 analytic: -1.137575, relative error: 5.284023e-08
numerical: 2.285663 analytic: 2.285662, relative error: 4.589834e-08
```

```
In [17]: # Now that we have a naive implementation of the softmax loss function and i
# implement a vectorized version in softmax_loss_vectorized.
# The two versions should compute the same results, but the vectorized versi
# much faster.
tic = time.time()
loss_naive, grad_naive = softmax_loss_naive(W, X_dev, y_dev, 0.000005)
toc = time.time()
print('naive loss: %e computed in %fs' % (loss_naive, toc - tic))

from cs682.classifiers.softmax import softmax_loss_vectorized
tic = time.time()
loss_vectorized, grad_vectorized = softmax_loss_vectorized(W, X_dev, y_dev,
toc = time.time()
```

```

print('vectorized loss: %e computed in %fs' % (loss_vectorized, toc - tic))

# As we did for the SVM, we use the Frobenius norm to compare the two versions
# of the gradient.
grad_difference = np.linalg.norm(grad_naive - grad_vectorized, ord='fro')
print('Loss difference: %f' % np.abs(loss_naive - loss_vectorized))
print('Gradient difference: %f' % grad_difference)

```

naive loss: 2.421284e+00 computed in 0.055501s  
vectorized loss: 2.421284e+00 computed in 0.013152s  
Loss difference: 0.000000  
Gradient difference: 0.000000

```

In [18]: # Use the validation set to tune hyperparameters (regularization strength and
# learning rate). You should experiment with different ranges for the learning
# rates and regularization strengths; if you are careful you should be able
# to get a classification accuracy of over 0.35 on the validation set.
from cs682.classifiers import Softmax
results = {}
best_val = -1
best_softmax = None
learning_rates = [1e-7, 5e-7]
regularization_strengths = [2.5e4, 5e4]

#####
# TODO:
# Use the validation set to set the learning rate and regularization strength.
# This should be identical to the validation that you did for the SVM; save
# the best trained softmax classifier in best_softmax.
#####
# Extend the provided values with one more learning rate and a weaker
# regularization strength.
search_lrs = learning_rates + [2e-7]
search_regs = regularization_strengths + [1e4]

for lr in search_lrs:
    for reg in search_regs:
        softmax = Softmax()
        softmax.train(X_train, y_train, learning_rate=lr, reg=reg,
                      num_iters=1500, verbose=False)
        train_accuracy = np.mean(softmax.predict(X_train) == y_train)
        val_accuracy = np.mean(softmax.predict(X_val) == y_val)
        results[(lr, reg)] = (train_accuracy, val_accuracy)
        if val_accuracy > best_val:
            best_val = val_accuracy
            best_softmax = softmax

#####
#                                     END OF YOUR CODE
#####

# Print out results.
for lr, reg in sorted(results):
    train_accuracy, val_accuracy = results[(lr, reg)]
    print('lr %e reg %e train accuracy: %f val accuracy: %f' % (
        lr, reg, train_accuracy, val_accuracy))

```

```
print('best validation accuracy achieved during cross-validation: %f' % best
lr 1.000000e-07 reg 1.000000e+04 train accuracy: 0.353061 val accuracy: 0.36
5000
lr 1.000000e-07 reg 2.500000e+04 train accuracy: 0.329347 val accuracy: 0.34
6000
lr 1.000000e-07 reg 5.000000e+04 train accuracy: 0.306816 val accuracy: 0.32
0000
lr 2.000000e-07 reg 1.000000e+04 train accuracy: 0.359286 val accuracy: 0.37
2000
lr 2.000000e-07 reg 2.500000e+04 train accuracy: 0.318286 val accuracy: 0.33
7000
lr 2.000000e-07 reg 5.000000e+04 train accuracy: 0.303673 val accuracy: 0.32
8000
lr 5.000000e-07 reg 1.000000e+04 train accuracy: 0.357143 val accuracy: 0.36
3000
lr 5.000000e-07 reg 2.500000e+04 train accuracy: 0.326571 val accuracy: 0.34
4000
lr 5.000000e-07 reg 5.000000e+04 train accuracy: 0.313082 val accuracy: 0.32
6000
best validation accuracy achieved during cross-validation: 0.372000
```

```
In [19]: # evaluate on test set
# Evaluate the best softmax on test set
y_test_pred = best_softmax.predict(X_test)
test_accuracy = np.mean(y_test == y_test_pred)
print('softmax on raw pixels final test set accuracy: %f' % (test_accuracy,
softmax on raw pixels final test set accuracy: 0.358000
```

### Inline Question - True or False

It's possible to add a new datapoint to a training set that would leave the SVM loss unchanged, but this is not the case with the Softmax classifier loss.

**Answer: True**

*Explanation:* For the SVM,  $L_i = \sum_{j \neq y_i} \max(0, s_j - s_{y_i} + 1)$  is exactly 0 whenever the correct score beats every other score by at least 1, i.e.  $1(J) = 0$  for all  $j$ . So we can add a new point that the current  $W$  already classifies correctly with that margin; it contributes 0 to the summed data loss and 0 to the gradient.

For softmax,  $L_i = -\log\left(\frac{e^{z_{y_i}}}{\sum_{j=1}^K e^{z_j}}\right)$ . Because every  $e^{z_j} > 0$ , the probability

$p(y = y_i | x) < 1$  for any finite scores, so  $L_i > 0$  always. Every new point therefore adds a strictly positive amount to the softmax loss.

(Strictly, since  $L = \frac{1}{N} \sum_i L_i$  is an average, adding a zero-loss point rescales the SVM average by  $\frac{N}{N+1}$ . The key idea is that the new point itself contributes zero hinge loss but always a positive softmax loss.)

```
In [20]: # Visualize the learned weights for each class
w = best_softmax.W[:-1,:] # strip out the bias
w = w.reshape(32, 32, 3, 10)

w_min, w_max = np.min(w), np.max(w)

classes = ['plane', 'car', 'bird', 'cat', 'deer', 'dog', 'frog', 'horse', 'ship', 'truck']
for i in range(10):
    plt.subplot(2, 5, i + 1)

    # Rescale the weights to be between 0 and 255
    wimg = 255.0 * (w[:, :, :, i].squeeze() - w_min) / (w_max - w_min)
    plt.imshow(wimg.astype('uint8'))
    plt.axis('off')
    plt.title(classes[i])
```

